

Crossmark

TAREA 7

RECEIVED
dd Month yyyyREVISED
dd Month yyyy

Aplicación de Kernel con Convolución

Daniel González¹ ¹Facultad de Ingeniería, Universidad Autónoma de Querétaro, Querétaro, México
14/10/2025**E-mail:** dgonzalez16@alumnos.uaq.mx**Keywords:** Python, kernel, convolución, preprocesamiento, imagenes, filtros

Abstract

En este trabajo se presenta la aplicación de filtros basados en kernels a una imagen utilizando el método de convolución, con la librería OpenCV de Python como de forma manual. Se busca demostrar el funcionamiento de la operación de convolución y analizar su efecto sobre la imagen original al modificar los valores del kernel. Mediante la comparación entre ambos enfoques, se verificó la similitud en los resultados además de demostrar de manera clara como se aplican los filtros más allá de solo usar una función la cual nos permite resaltar bordes, suavizar o realzar detalles en imágenes digitales.

1 Introducción

El reconocimiento de patrones escritos a mano es uno de los problemas más comunes en el campo del aprendizaje automático, ya que representa un reto tanto por la variabilidad de la escritura humana es decir la caligrafia unica asi como por la necesidad de modelos eficientes. Uno de los conjuntos de datos más utilizados para abordar este problema es MNIST (Modified National Institute of Standards and Technology), el cual contiene 70,000 imágenes en escala de grises de dígitos manuscritos del 0 al 9, divididas en 60,000 para entrenamiento y 10,000 para prueba.

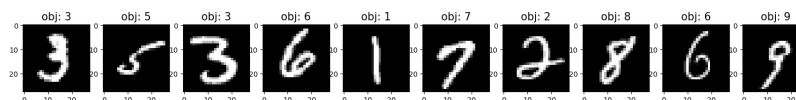


Figure 1. Ejemplos de numeros de la Mnist

Cada imagen tiene un tamaño de 28×28 píxeles y constituye un estándar para la evaluación de algoritmos de clasificación de imágenes.

Entre los modelos más sencillos para abordar esta tarea se encuentra el perceptrón multicapa (MLP), una red neuronal artificial en la cual cada neurona de una capa se conecta con todas las de la siguiente. En este caso se implementó un perceptrón con una capa oculta y una capa de salida, entrenado sobre el dataset MNIST, para la tarea de clasificación de numeros. A pesar de ser una arquitectura simple, el perceptrón es capaz de aprender representaciones útiles que permiten un buen desempeño con este problema.

Para probar nuestro modelo, se diseñó además una interfaz gráfica en Python utilizando la librería **Tkinter**. Esta interfaz permite al usuario dibujar un numero en una ventana y recibir la predicción del modelo, integrando así el entrenamiento de la red neuronal con una aplicación interactiva. De igual manera, se exploraron técnicas de preprocesamiento de la imagen, como el escalado, centrado y normalización, con el objetivo de mejorar la correspondencia entre los dígitos dibujados por el usuario y aquellos contenidos en el dataset de entrenamiento.

2 Desarrollo

2.1 Modelo

Se implementó una red perceptron con entrada de 784 ya que al tener imagenes de 28×28 sera nuestra entrada de datos 'Aplanados', además de esto se eligio como capa oculta 128 neuronas como numero promedio entre aprendizaje y tiempo de entrenamiento asi como una función de activacion ReLU para que aprenda patrones no lineales. Por ultimo una salida de 10, que seran

nuestros numeros del 0 al 9 con función de activacion softmax para tener la probabilidad segun el modelo. Se usó el optimizador Adam, muy eficiente y ampliamente utilizado en problemas de clasificación. La función de pérdida *Sparse Categorical Crossentropy* es la adecuada cuando las etiquetas (`y_train`) son enteros (0–9) y no vectores one-hot. Por último, se eligieron 5 épocas para que un perceptrón aprenda MNIST sin sobreajustar demasiado. El *batch size* de 128 representa un tamaño estándar que balancea memoria y velocidad, mientras que el parámetro `validation_split` con valor de 0.1 indica que el 10% de los datos de entrenamiento se reservan para validación, monitoreando el rendimiento en datos no vistos durante cada época.



Figure 2. Parametros del Modelo Usado

Guardamos el modelo para no tener que entrenarlo cada vez que se corra el codigo y pasamos a la parte de diseñar nuestra interfaz.

2.2 Interfaz

Para el desarrollo de la interfaz gráfica en Python se uso la librería `Tkinter`. Esta interfaz tiene como objetivo permitir la interacción directa del usuario con el sistema de reconocimiento de numeros.

La aplicación presenta un lienzo digital de fondo negro sobre el cual el usuario puede dibujar un dígito mediante el cursor, utilizando un trazo blanco de grosor ajustable. Paralelamente, se emplea la librería `PIL` para almacenar el dibujo en una imagen procesable, lo que permite su posterior transformación al formato compatible con el modelo.



Figure 3. Interfaz Diseñada

El preprocesamiento implementado consiste en escalar y centrar el dígito dentro de una imagen de 28×28 píxeles, normalizar sus valores de intensidad y adaptarlo a la entrada esperada por el perceptrón. Una vez realizado este proceso, al presionar el botón *Predecir*, la imagen es pasada al modelo previamente entrenado, mostrando en pantalla el resultado de la clasificación junto con la probabilidad asociada.



Figure 4. Predicción del Numero 7

La interfaz incluye también un botón *Borrar*, que reinicia el lienzo para permitir múltiples intentos. De esta forma, se integran el entrenamiento del modelo y su uso práctico en tiempo real, proporcionando una herramienta sencilla e interactiva para la demostración del reconocimiento automático de dígitos escritos a mano.

Durante el desarrollo se enfrentaron problemas con este preprocesamiento ya que aunque nuestro modelo se entreno correctamente y nos mostraba buenos valores de accuracy la prediccion no era la mejor, debido a esto se implemento otro boton *Ver 28×28* que nos permite ver la entrada que le llega a nuestro modelo, de esta manera nos aseguramos de ver lo mismo que el modelo en caso de querer hacer un chequeo manual.

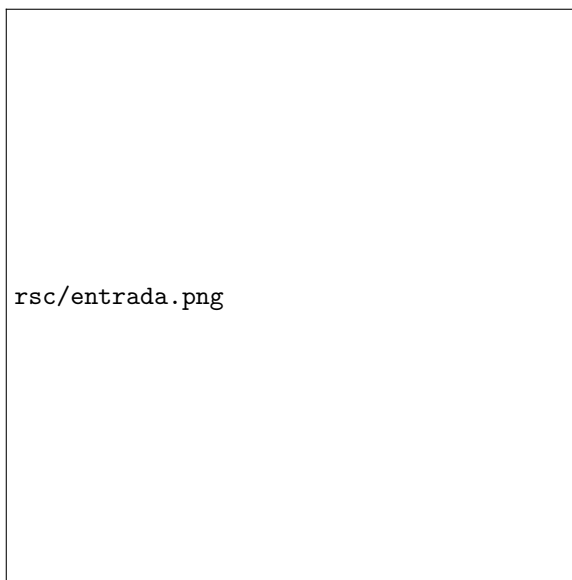


Figure 5. Entrada del modelo del numero 7 visto en la Figura 4

3 Conclusión

El desarrollo de este proyecto permitió comprobar que un perceptrón multicapa, aun con una arquitectura sencilla, demuestra un desempeño notable para la de clasificación de dígitos manuscritos utilizando el conjunto de datos MNIST. La integración con una interfaz gráfica demostró la utilidad práctica del modelo, ya que brindó la posibilidad de interactuar con el sistema así como comprobar en tiempo real sus predicciones. Durante el proceso se demostró la importancia del preprocesamiento de las imágenes para lograr una correspondencia adecuada entre los datos de entrenamiento y los dígitos dibujados por el usuario, lo cual refleja un aspecto clave para mejorar la robustez del sistema.